SIMPLE BANK MANAGEMENT SYSTEM

A Console-Based Application using Python & OOP

Prepared By: Ritesh Chaudhary

Contents

1. Introduction **2**
2. Project Objectives **2**
3. Project Scope **2**
4. Technologies and Tools Used **3**
5. System Requirements **3**
5.1 Functional Requirements 3
5.2 Input Validation Requirements 3
6. System Architecture and Program Design **4**
6.1 Class Structure 4
6.2 Main Program Components 4
7. Object-Oriented Programming Concepts Implemented **5**
7.1 Abstraction 5
7.2 Encapsulation 5
7.3 Inheritance 6
7.4 Polymorphism 6
7.5 Constructors 6
7.6 Private and Protected Access Modifiers 7
8. Custom Exception Handling **7**
9. Account Number Generation and Account Management **7**
10. Application Workflow **8**
11. Sample Console Demonstration **8**
11.1 Main Menu 8
11.2 Creating an Account 9
11.3 Depositing Money 9
11.4 Withdrawing Money 9
11.5 Viewing Account Details 10
11.6 Handling Invalid Account Numbers 10
11.7 Handling Invalid Money Input 10
11.8 Handling Insufficient Balance 10
12. Testing and Expected Results **11**
13. Limitations **11**
14. Future Enhancements **12**
10. Conclusion **12**

1. Introduction

The Simple Bank Management System is a console-based Python application developed to demonstrate the practical implementation of Object-Oriented Programming (OOP).

The application provides basic banking operations, including creating accounts, depositing money, withdrawing money, and viewing account details. Each account is assigned a randomly generated 16-digit account number.

The project focuses on object-oriented design, input validation, custom exception handling, and organizing program functionality into reusable classes and functions.

2. Project Objectives

The main objectives of this project are:

- Develop a basic banking application using Python.
- Apply OOP concepts such as abstraction, encapsulation, inheritance, and polymorphism.
- Create and manage multiple bank accounts.
- Generate random 16-digit account numbers.
- Implement deposit and withdrawal operations.
- Validate user input and prevent invalid transactions.
- Handle application errors using custom exceptions.
- Organize the program into reusable classes and functions.
- Demonstrate structured programming and basic application-level with error handling.

3. Project Scope

The application provides the following functionalities:

Feature	**Description**
Create Account	Creates a new bank account using the customer's name and starting balance.
Account Number Generation	Generates a random 16-digit account number.
Deposit Money	Adds a valid amount to an existing account balance.
Withdraw Money	Deducts money if the account has sufficient balance.
View Account Details	Displays the account holder's name, account number, and current balance.
Input Validation	Checks names, account numbers, and money amounts before processing.
Exception Handling	Handles invalid inputs, missing accounts, insufficient balance, and duplicate account numbers.

Main Menu	Provides a menu-driven interface for accessing the application features.

4. Technologies and Tools Used

Technology / Module	**Purpose**
Python	Main programming language
Random Module	Generates 16-digit account numbers
ABC Module	Implements abstraction using abstract base classes and abstract methods.
OOP	Structures the application using classes, objects, inheritance, encapsulation, and polymorphism.
Python List	Stores multiple bank account objects during program execution.
Custom Exceptions	Handles application-specific validation and operational errors.
Console / Terminal	Provides the user interface for interacting with the application.

5. System Requirements

5.1 Functional Requirements

The system provides five main menu options:

1. Create Account
2. Deposit Money
3. Withdraw Money
4. View Details
5. Exit

Each operation is handled through separate functions, while account-related operations are implemented through classes and methods.

5.2 Input Validation Requirements

The application applies the following validation rules:

Account Holder Name

- The name cannot be empty.
- Only alphabetic characters and spaces are accepted.
- Invalid characters are rejected.

Account Number

- The account number must contain exactly 16 digits.
- Only numeric input is accepted.
- The account number must match an existing account before an account operation can be performed.

Money Amount

- Starting balance, deposit amount, and withdrawal amount must be positive integers.
- Zero and negative values are not accepted.
- Decimal values and non-numeric input are rejected.
- A withdrawal cannot exceed the available account balance.

6. System Architecture and Program Design

The application follows an object-oriented structure in which account-related information and operations are organized into classes.

6.1 Class Structure

The program contains two main classes:

BankAccount (Abstract Base Class)

This class acts as the parent class and provides the common structure for bank accounts.

Its responsibilities include:

- Initialising account information.
- Validating account holder names and starting balances.
- Generating account numbers.
- Maintaining private account attributes.
- Providing getter methods.
- Defining abstract methods for deposit, withdrawal, and account details.

Bank (Child Class)

The Bank class inherits from BankAccount and implements its abstract methods.

Its responsibilities include:

- Depositing money into an account.
- Withdrawing money from an account.
- Checking the available balance.
- Displaying account information.

6.2 Main Program Components

Apart from the two classes, the application uses separate functions to manage the overall workflow:

- validate_name() – validates the account holder's name.
- validate_account_number() – validates the account number format.
- validate_amount() – validates money amounts.
- create_unique_account() – creates an account and checks for duplicate account numbers.
- find_account() – searches for an account in the account list.
- create_account() – handles account creation.
- get_valid_account_number() – receives and validates account number input.
- get_valid_amount() – receives and validates money input.
- deposit_money() – handles deposit operations.
- withdraw_money() – handles withdrawal operations.
- view_details() – displays account information.

The main menu runs inside a while loop, allowing users to perform multiple operations until they select the Exit option.

7. Object-Oriented Programming Concepts Implemented

One of the main purposes of this project is to demonstrate the practical implementation of Object-Oriented Programming concepts.

7.1 Abstraction

Abstraction is implemented using Python's ABC module and the ABC base class.

The BankAccount class is declared as an abstract class. It defines three abstract methods:

- deposit()
- withdraw()
- show_details()

These methods define the operations that a concrete bank account class must implement.

The Bank class inherits from BankAccount and provides the actual implementation of these methods.

This separates the general structure of an account from the implementation of its banking operations.

7.2 Encapsulation

Encapsulation is implemented by keeping account information inside the BankAccount class.

The following attributes are declared as private:

- __name

- __balance
- __account_number

Getter methods are provided to access these attributes:

- get_name()
- get_balance()
- get_account_number()

A protected method, _set_balance(), is used internally by the banking operations to update the account balance.

This approach helps control access to account information and keeps the account-related data organised within the class.

7.3 Inheritance

Inheritance is implemented through the relationship between the BankAccount and Bank classes.

The Bank class inherits from BankAccount and uses super().__init__() to initialise the parent class attributes.

This avoids repeating the common account initialisation logic and allows the child class to reuse the functionality provided by the parent class.

7.4 Polymorphism

Polymorphism is demonstrated through method overriding.

The abstract methods declared in BankAccount are implemented in the Bank class with their own behaviour.

For example:

- deposit() implements the deposit operation.
- withdraw() implements the withdrawal operation.
- show_details() displays the account information.

The child class provides the actual behaviour while following the method structure defined by the abstract parent class.

7.5 Constructors

The __init__() method is used as a constructor to initialise an account object.

When a new Bank object is created, the constructor:

- Receives the account holder's name.
- Receives the starting balance.
- Validates the supplied information.

- Generates a random 16-digit account number.

The super() function is used to call the parent class constructor.

7.6 Private and Protected Access Modifiers

Python uses naming conventions to represent different levels of attribute access.

This project uses:

- Double underscore (__) for private attributes.
- Single underscore (_) for a protected-by-convention method.

These conventions help organise access to account information and internal operations.

8. Custom Exception Handling

The application uses custom exception classes to handle specific errors instead of relying only on general Python exceptions.

The following custom exceptions are implemented:

Custom Exception	**Purpose**
InvalidAmountError	Raised when a money amount is invalid, zero, negative, or not an integer.
InsufficientBalanceError	Raised when the withdrawal amount exceeds the available balance.
AccountNotFoundError	Raised when the supplied account number does not match an existing account.
DuplicateAccountError	Raised when a generated account number already exists.
InvalidNameError	Raised when the account holder's name is empty or contains invalid characters.
InvalidAccountNumberError	Raised when an account number has an invalid type or format.

The program uses try and except blocks to catch these exceptions and display understandable error messages.

For example, if a user attempts to withdraw more money than the available balance, the application raises an InsufficientBalanceError and displays an appropriate message instead of performing the withdrawal.

The application also handles unexpected exceptions separately to prevent common input errors from interrupting the normal menu workflow.

9. Account Number Generation and Account Management

Each new account receives a randomly generated 16-digit account number using Python's random module.

The account number is generated within the following range:

10**15 to 10**16 - 1

The application stores account objects inside a list named accounts.

Before adding a new account, the create_unique_account() function checks whether the generated account number already exists in the list.

The find_account() function searches through the account list and returns the matching account object. If no matching account is found, it raises an AccountNotFoundError with the message:

Account number does not match

This allows deposit, withdrawal, and account-detail operations to access the correct account.

10. Application Workflow

The overall application workflow is as follows:

1. The program starts and displays the main menu.
2. The user selects an operation.
3. For account creation, the program collects the account holder's name and starting balance.
4. The supplied information is validated.
5. A new account object is created and assigned a random 16-digit account number.
6. The account object is stored in the account list.
7. For deposit, withdrawal, or viewing details, the user provides an account number.
8. The program validates the account number and searches the account list.
9. If the account exists, the selected operation is performed.
10. If the account does not exist or the input is invalid, an appropriate error message is displayed.
11. The program returns to the main menu.
12. The application terminates when the user selects Exit or interrupts the program.

11. Sample Console Demonstration

The following examples illustrate how a user can interact with the application.

Note: The account number and values below are illustrative. Actual account numbers are generated randomly when the program runs.

11.1 Main Menu

========== BANK SYSTEM ==========

1. Create Account

2. Deposit Money

3. Withdraw Money

4. View Details

5. Exit

==========

Enter your choice:

11.2 Creating an Account

Enter your choice: 1

Enter account holder name: Ritesh Chaudhary

Enter starting money: 10000

Account created successfully!

Name : Ritesh Chaudhary

Account Number : 1234567890123456

Balance : Rs.10000.00

11.3 Depositing Money

Enter your choice: 2

Enter account number: 1234567890123456

Enter amount to deposit: 5000

Money deposited successfully.

New Balance: Rs.15000.00

11.4 Withdrawing Money

Enter your choice: 3

Enter account number: 1234567890123456

Enter amount to withdraw: 2000

Money withdrawn successfully.

New Balance: Rs.13000.00

11.5 Viewing Account Details

Enter your choice: 4

Enter account number: 1234567890123456

----- Account Details -----

Name : Ritesh Chaudhary

Account Number : 1234567890123456

Balance : Rs.13000.00

11.6 Handling Invalid Account Numbers

Enter your choice: 2

Enter account number: 1111111111111111

Error: Account number does not match

11.7 Handling Invalid Money Input

Enter amount to deposit: 100abc

Error: Amount must contain only integer digits.

11.8 Handling Insufficient Balance

Enter amount to withdraw: 50000

Error: Withdraw amount must not exceed available balance.

12. Testing and Expected Results

The following test cases describe the expected behaviour of the application based on its implemented validation rules.

Test Case	Input / Condition	Expected Result
1	Enter a valid name and positive starting balance.	Account is created successfully.
2	Enter an empty account holder name.	InvalidNameError is raised.
3	Enter a name containing numbers or special characters.	Name validation error is displayed.
4	Enter a positive integer deposit amount.	Account balance increases.
5	Enter zero or a negative deposit amount.	InvalidAmountError is raised.
6	Enter a decimal or non-numeric money amount.	Invalid amount message is displayed.
7	Withdraw an amount within the available balance.	Account balance decreases.
8	Withdraw more than the available balance.	InsufficientBalanceError is raised.
9	Enter an account number with fewer or more than 16 digits.	InvalidAccountNumberError is raised.
10	Search using an account number that does not exist.	Account number mismatch message is displayed.
11	View details using a valid account number.	Account information is displayed.
12	Select an invalid menu option.	Invalid menu choice message is displayed.
13	Select the Exit option.	Application terminates successfully.

13. Limitations

The current version is designed as a learning and demonstration project. It has the following limitations:

- **Console-Based Interface:** The application operates through a terminal and does not provide a graphical user interface or web interface.
- **Temporary Data Storage:** Account information is stored in a Python list and is available only while the program is running.
- **No Database Integration:** The application does not use a database such as MySQL, PostgreSQL, or SQL Server.
- **No User Authentication:** The current version does not include customer login, PIN verification, or account access permissions.
- **Basic Account Number Generation:** Account numbers are generated using Python's random module for demonstration purposes and are not suitable for production banking security.
- **Integer-Based Transactions:** The application accepts whole-number money amounts rather than decimal currency values.

14. Future Enhancements

The application can be extended with additional functionality in future development:

- Integrate a relational database to store account information permanently.
- Develop a graphical user interface or web-based banking application.
- Implement customer authentication and secure account access.
- Add transaction history and account statements.
- Introduce transaction dates and unique transaction IDs.
- Add account update and deletion functionality.
- Implement logging and more detailed operational error tracking.
- Improve account number generation and security for a more realistic banking application.

10. Conclusion

The Simple Bank Management System demonstrates the practical application of Python programming and Object-Oriented Programming concepts through a basic banking application.

The project implements account creation, deposit, withdrawal, and account-detail operations using classes, objects, and reusable functions. It also demonstrates abstraction, encapsulation, inheritance, polymorphism, constructors, private attributes, custom exceptions, and input validation.

Developing this project provided practical experience in structuring a Python application, managing objects, validating user input, handling application errors, and applying OOP principles to a real-world-inspired problem.

Although the current implementation is a console-based demonstration and does not include permanent database storage or production-level security, it provides a foundation for developing more advanced software applications.

End of Project Documentation.